

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



CineCraft :

Sistema de Gestión de Reseñas de Películas con GitFlow

PONDERACIÓN: 20 pts



Tiempo estimado: 30 hrs

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje.....	4
2.1. Objetivo SMART.....	4
3. Enunciado de la Práctica.....	5
3.1 Descripción del problema a resolver.....	5
3.2 Alcance de la práctica.....	5
4. Entregables.....	8
5. Material de apoyo.....	8
6. Recursos y herramientas a utilizar.....	9
7. Cronograma.....	10
8. Rúbrica de Calificación.....	10
8.1 Requisitos para optar a la calificación.....	10
8.2 Resumen de Puntuaciones.....	11
Detalle de la Calificación.....	13
Comentarios Generales.....	24

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Trabajo en Equipo	Se fomenta la colaboración entre estudiantes mediante el uso de GitFlow, donde cada integrante debe contribuir activamente al desarrollo de features específicas y coordinarse para integrar sus cambios de manera efectiva.
Responsabilidad	Cada estudiante es responsable de sus commits, el desarrollo de sus features asignadas y de seguir las buenas prácticas en el control de versiones, asegurando la calidad del código que integra al proyecto.

1.2. Competencia(s)

Con la elaboración de esta práctica usted adquirirá las siguientes competencias:

Tipo de Competencia	
Competencia General	Desarrollar soluciones de software utilizando metodologías, herramientas y buenas prácticas de la ingeniería de software que permitan el trabajo colaborativo y el control efectivo de versiones en proyectos de desarrollo.
Competencia Específica	Aplicar GitFlow como estrategia de branching en el desarrollo de aplicaciones web, gestionando efectivamente las ramas de desarrollo, releases y hotfixes para mantener un flujo de trabajo organizado y profesional.

1.3. Habilidad(es) blandas a formar

La práctica le permitirá desarrollar las siguientes habilidades:

Habilidad	Individual / Grupal	Descripción
Comunicación efectiva	Grupal	Los integrantes deberán coordinar tareas, revisar el trabajo de sus compañeros y resolver conflictos de integración de código, lo que exige una comunicación clara y oportuna dentro del equipo.

Gestión del tiempo	Individual	Cada estudiante deberá planificar el desarrollo de su(s) feature(s) dentro del plazo establecido, realizando commits regulares que evidencien avance continuo y no trabajo de último momento.
Pensamiento crítico	Individual	Al diseñar la arquitectura de la aplicación y tomar decisiones sobre tecnologías, estructura de la base de datos y flujo de branching, el estudiante deberá evaluar opciones y justificar sus elecciones técnicas.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Los estudiantes desarrollarán una aplicación web de gestión de reseñas de películas implementando GitFlow como modelo de control de versiones, incluyendo al menos 6 funcionalidades principales (agregar, modificar, eliminar, destacar, archivar y compartir reseñas).	Realizar al menos 2 releases a la rama Main con un mínimo de 6 features completamente funcionales, con commits documentados y un diagrama de workflow completo que refleje todas las operaciones realizadas.	Utilizando Git como sistema de control de versiones, frameworks y lenguajes de programación a elección del equipo.	Preparando a los estudiantes para trabajar en proyectos reales de desarrollo de software donde el control de versiones y el trabajo colaborativo son habilidades fundamentales en el ámbito profesional.	Completar el desarrollo y entrega de la práctica en un plazo máximo de 2 semanas, con entregas parciales de features y releases según el cronograma establecido.

3. Enunciado de la Práctica

3.1 Descripción del problema a resolver

CineCraft es un sistema que se desarrollará para la gestión de reseñas de películas. El objetivo de la aplicación es permitir al usuario registrar, organizar y compartir sus opiniones sobre películas de forma estructurada e intuitiva.

El sistema debe permitir crear, modificar y eliminar reseñas, organizarlas por género o etiquetas personalizadas, destacar las favoritas y archivar aquellas de menor relevancia. Además, los usuarios podrán compartir reseñas con otros usuarios de la plataforma.

Un aspecto fundamental de esta práctica es la implementación de GitFlow como estrategia de branching, lo que permitirá a los estudiantes experimentar con un flujo de trabajo profesional utilizado en la industria del desarrollo de software. Cada funcionalidad deberá desarrollarse en una rama feature independiente, seguir el proceso de integración a develop y finalmente llegar a producción mediante releases controlados.

3.2 Alcance de la práctica

Obligatorio:

- Implementación de GitFlow con ramas Main/Master, Develop, Features, Release y Hotfix
- Desarrollo de las siguientes funcionalidades:
 - **Agregar Reseña:** Permite registrar una nueva reseña con los siguientes datos:
 - Título de la película
 - Calificación (1–5 estrellas)
 - Comentario/opinión
 - Etiqueta/género.

Se debe validar que el título de la película no vaya vacío. Las etiquetas pueden crearse en el momento de agregar una nueva reseña, o seleccionarse de las ya existentes en reseñas anteriores.
 - **Eliminar Reseña:** Permite remover una reseña de la lista. Se recomienda que antes de realizar esta acción, el usuario pueda confirmar su decisión de eliminar la reseña.
 - **Modificar Reseña:** Permite editar cualquier campo de una reseña previamente registrada (título, calificación, comentario y etiqueta), respetando las mismas validaciones aplicadas al momento del alta.

- **Destacar Reseña:** Marca una reseña como favorita o destacada, moviéndola al apartado "Destacadas" en la página principal. Se puede desmarcar en cualquier momento.
- **Archivar Reseña:** Permite conservar reseñas de menor relevancia sin eliminarlas, moviéndolas a la sección "Archivadas". La reseña archivada deja de aparecer en el dashboard principal, únicamente en la sección de Archivadas, y puede desarchivarse posteriormente.
- **Compartir Reseñas entre usuarios:** Permite que un usuario comparta una reseña con otro usuario registrado en la plataforma. La reseña compartida aparece automáticamente en el dashboard de notificaciones del destinatario con un indicador visual y el nombre del propietario. El propietario puede ver en una sección "Compartidas" todas las reseñas que ha compartido y con quiénes.
- **Manejo de usuarios:** La aplicación debe permitir registrar perfiles de usuarios, validar que el login sea a la cuenta asociada y permitir modificar la información personal.
- Almacenamiento de datos en base de datos (local o en la nube)
- Realizar al menos 2 releases a la rama Main con etiquetado de versiones (versión final mínima: v2.0.0)
- Realizar al menos un Hotfix
- Aplicar buenas prácticas en los commits y nomenclatura de ramas
- Validaciones de campos obligatorios (el título de la película no puede estar vacío)

MANEJO DE BRANCHES

Main/Master y Develop

Estas ramas se manejan de la siguiente manera:

- **Main/Master:** es la rama principal del repositorio, en esta rama se maneja un entorno de producción, es decir, solo se tendrá el código funcional.
- **Develop:** Esta rama se crea a partir de la rama Main/Master, de esta rama se crearán las ramas features. En esta se mantendrá el código antes de realizarse un release a la rama Main

Release

Esta rama nace de la rama develop y se utiliza para pasar los cambios de una nueva versión a la rama main. Al fusionar la rama release con la rama main se solicita que se agregue una etiqueta a la rama main, con el nombre de la versión. (Al finalizar se espera que se tenga la versión 2.0.0). Asimismo, la rama release también se debe fusionar con la rama develop.

Hotfix

Nace de la rama main para corregir un error crítico en producción, al finalizar la corrección, esta rama se fusiona tanto en la rama develop como en la rama main, y en la rama main se crea una etiqueta con el nuevo número de versión.

Features

Para cada una de las funcionalidades se debe crear una rama con la siguiente nomenclatura "Feature/FeatureName_#carnet", donde FeatureName es el nombre de la funcionalidad a incorporar y Id el número de carnet. Al finalizar cada feature se debe fusionar a la rama develop.

3.3 Requerimientos técnicos

Los estudiantes tienen libertad para elegir las tecnologías que consideren más apropiadas para el desarrollo de la práctica, sin embargo, deben cumplir con los siguientes requerimientos técnicos fundamentales:

Control de Versiones:

- Uso obligatorio de Git como sistema de control de versiones
- Implementación estricta de GitFlow como estrategia de branching
- Nomenclatura de ramas features: "Feature/FeatureName_#Carnet"
- Commits descriptivos siguiendo buenas prácticas

Backend y Frontend:

- El lenguaje de programación, frameworks y librerías quedan a discreción del grupo
- La aplicación debe ser una aplicación web funcional
- Debe implementarse una API o sistema de comunicación entre frontend y backend

Base de Datos:

- Los datos de las notas deben almacenarse en una base de datos
- Puede ser local o en la nube (a elección del grupo)
- Debe mantener la persistencia de información entre sesiones

Documentación:

- Diagrama de workflow en girKraken u otra herramienta de visualización similar. El diagrama debe incluir todas las ramas, merges y acciones realizadas
- Cada acción debe estar documentada con el número de carnet del estudiante responsable

Colaboración:

- El auxiliar debe ser agregado como colaborador en el repositorio
 - Sección A:
 - Sección B:

4. Entregables

Tipo	Descripción
Repositorio en GitHub	Repositorio con el nombre -AYD1-Practica1S2026_<Sección>_G# (donde # es el número de grupo) que contenga todo el código fuente de la aplicación, correctamente organizado según GitFlow.
Aplicación Web Funcional	Implementación completa de CineCraft con todas las funcionalidades requeridas.
Manual Técnico	Documento en formato Markdown con la descripción de los métodos utilizados, los requerimientos de la aplicación y observaciones técnicas relevantes y ERD de la base de datos utilizada.
Manual de Usuario	Documento en formato Markdown con instrucciones claras para interactuar con el sistema, incluyendo ejemplos de uso y capturas de pantalla.
Entrega UEDI	Link del repositorio de GitLab entregado a través de la plataforma UEDI antes de la fecha límite.

5. Material de apoyo

Los estudiantes pueden consultar los siguientes recursos para la elaboración de la práctica:

GitFlow y Control de Versiones:

- Documentación oficial de Git: <https://git-scm.com/doc>
- Atlassian Git Tutorial - GitFlow Workflow: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- Vincent Driessen - "A successful Git branching model": <https://nvie.com/posts/a-successful-git-branching-model/>
- Pro Git Book (Español): <https://git-scm.com/book/es/v2>

Desarrollo Web:

- MDN Web Docs (Mozilla Developer Network): <https://developer.mozilla.org/>
- W3Schools - Tutorial de desarrollo web: <https://www.w3schools.com/>
- FreeCodeCamp - Tutoriales gratuitos: <https://www.freecodecamp.org/>

Bases de Datos:

- Documentación de MySQL: <https://dev.mysql.com/doc/>
- Documentación de PostgreSQL: <https://www.postgresql.org/docs/>
- MongoDB University: <https://university.mongodb.com/>

6. Recursos y herramientas a utilizar

Software obligatorio:

- Git (sistema de control de versiones)
- GitHub (plataforma de repositorios)

Software a elección del grupo:

- IDE de desarrollo (VS Code, IntelliJ IDEA, WebStorm, etc.)
- Lenguajes de programación para backend (Node.js, Python, Java, PHP, etc.)
- Frameworks frontend (React, Angular, Vue.js, etc.)
- Sistema de gestión de base de datos (MySQL, PostgreSQL, MongoDB, etc.)
- Herramientas de testing y debugging

Plataformas:

- GitHub o GitLab (repositorio principal)
- UEDI (para entrega oficial)
- Servicios de hosting opcional (Heroku, Vercel, AWS, etc.)

Lecturas recomendadas:

- Documentación oficial de las tecnologías seleccionadas
- Artículos sobre buenas prácticas en control de versiones
- Guías de arquitectura de aplicaciones web
- Patrones de diseño de software

7. Cronograma

El cronograma describe las etapas clave de la práctica, los plazos estimados para cada una y el proceso de asignación, elaboración y calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Práctica	03/08/2026	03/02/2026
Elaboración	03/08/2026	17/08/2026
Calificación	22/08/2026	22/08/2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El desarrollo debe implementar GitFlow como estrategia de branching obligatoria con las ramas Main/Master, Develop, Features, Release y Hotfix.	
Uso de herramientas requeridas	Se debe evidenciar el uso correcto de Git, repositorio en GitHub/GitLab.	

Gestión y entregas de práctica	El repositorio debe tener el nombre correcto -AYD1-Practica1S2026_<Sección>_G# (donde # es el número de grupo) y contener todo el código fuente y el diagrama de workflow en GitKraken u otra herramienta de visualización similar. Se deben haber realizado al menos 2 releases a Main.	
Documentación obligatoria	Se debe incluir Manual Técnico, Manual de Usuario.	
Pruebas y funcionalidad mínima	La aplicación debe contar con las funcionalidades principales completamente operativas: agregar, modificar, eliminar. Los datos deben persistir en la base de datos.	
Presencia en calificación	Todos los integrantes del grupo deben estar presentes el día de la calificación. Se validará la participación individual mediante actividades y preguntas.	
Colaborador agregado	El auxiliar de la sección correspondiente debe estar agregado como colaborador en el repositorio con permisos de lectura.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Configuración Inicial del Repositorio	4	
Conventional Commits	6	
Flujo Visual de Ramas	3	
Ramas Feature	4	
GitFlow (Releases)	16	
Flujo HotFix	12	
Manual Técnico con ERD	3	
Manual de Usuario	3	
Sub-Total Habilidades	51	
2. Conocimiento		
Manejo de Cuentas	12	
Gestión de Reseñas	15	
Funcionalidades Avanzadas de Reseñas	10	
Colaboración entre usuarios	8	
Preguntas	4	
Sub-Total Conocimiento	49	
TOTAL	100	

Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	51				
1.1	CONFIGURACIÓN INICIAL DEL REPOSITORIO	4				
1.1.1	Creación de rama Develop	2	Descripción	La rama develop está creada correctamente a partir de main/master. Es evidente que fue creada al inicio del proyecto. Se usó como rama base para crear todas las features. Sigue la estructura de GitFlow correctamente.	No existe rama develop, fue creada incorrectamente, no se usó como base para features o no sigue la estructura de GitFlow.	
			Rango del punteo	2 - 1.2	1.1 - 0	
	Archivo .gitignore		Descripción	Existe un archivo .gitignore apropiado para el proyecto. Excluye archivos innecesarios (node_modules, archivos de configuración local, archivos de IDE, etc.). Está correctamente configurado desde el inicio del proyecto. No hay archivos que deberían estar ignorados en el repositorio.	No existe .gitignore, está mal configurado, hay archivos que deberían estar ignorados en el repositorio o fue agregado tardíamente.	

			Rango del punteo	2 - 1.2	1.1 - 0	
1.2	CONVENTIONAL COMMITS	6				
1.2.1	Conventional Commits Correctos	6	Descripción	Los commits siguen el formato de Conventional Commits (feat:, fix:, docs:, style:, refactor:, test:, chore:). Los mensajes son descriptivos y claros. Describen qué se hizo y por qué. Están en tiempo presente. Hay commits regulares durante el desarrollo (no todo en commits masivos). Cada integrante tiene commits individuales identificables.	Los commits no siguen Conventional Commits, los mensajes son genéricos o confusos ("update", "changes", "fix"), no son descriptivos, hay evidencia de commits masivos al final del proyecto, o falta evidencia de commits individuales por integrante.	
			Rango del punteo	6 - 3.7	3.6 - 0	
1.3	FLUJO VISUAL DE RAMAS	3				
1.3.1	Flujo Visual (GitKraken)	3	Descripción	Existe un diagrama visual del flujo de trabajo creado con GitKraken u otra herramienta profesional. El diagrama muestra claramente todas las ramas (main, develop, features, releases, hotfix). Muestra todos los merges realizados. Las acciones están documentadas con el número de carnet del responsable. El diagrama es claro, organizado y refleja fielmente el historial del repositorio.	No existe diagrama, está incompleto, le faltan ramas o merges, no documenta responsables, no corresponde con el historial real del repositorio o la presentación es confusa/poco profesional.	

			Rango del punteo	3 - 1.8	1.7 - 0	
1.4	RAMAS FEATURE	4				
1.4.1	Nomenclatura Correcta	1	Descripción	Todas las ramas feature siguen estrictamente la nomenclatura "Feature/FeatureName_#Carnet". Los nombres son descriptivos de la funcionalidad desarrollada. El número de carnet está presente en todas las ramas feature.	Las ramas no siguen la nomenclatura establecida, faltan números de carnet, los nombres no son descriptivos o hay inconsistencias en el formato.	
			Rango del punteo	1 - 0.6	0.6 - 0	
	Ramas por cada integrante		Descripción	Cada integrante del grupo creó al menos una rama feature con su número de carnet. Hay evidencia de trabajo distribuido entre los miembros. Cada integrante tiene commits en su(s) propia(s) rama(s) feature. Las features están balanceadas en complejidad entre integrantes.	No todos los integrantes tienen ramas feature, hay integrantes sin commits en features, el trabajo no está distribuido equitativamente o hay evidencia de que solo una persona hizo todo.	
1.4.2		3	Rango del punteo	3 - 1.8	1.7 - 0	
1.5	GITFLOW - RELEASES	16				
1.5.1	Merge Feature Into Develop Correctos	4	Descripción	Todas las ramas feature se fusionaron correctamente a develop. Los merges no generaron conflictos sin resolver. Se siguió el orden lógico de desarrollo.	Las features no se fusionaron a develop, se fusionaron directamente a main, hay conflictos sin resolver o el orden de merges no sigue una lógica.	

				Cada merge tiene un commit descriptivo.		
			Rango del punteo	4 - 2.4	2.3 - 0	
1.5.2	Merge Release Into Main Correctos	4	Descripción	Al menos 2 ramas release se fusionaron correctamente a main. Los merges representan versiones estables del proyecto. Se realizaron en momentos apropiados del desarrollo. Cada merge tiene documentación del contenido de la release.	Menos de 2 releases a main, las releases no se fusionaron correctamente, se fusionaron versiones inestables o falta documentación de las releases.	
			Rango del punteo	4 - 2.4	2.3 - 0	
1.5.3	Merge Release Into Develop Correctos	4	Descripción	Las ramas release también se fusionaron de vuelta a develop después de fusionarse a main. Esto mantiene develop actualizado con los cambios de producción. Los merges se realizaron correctamente sin conflictos.	Las releases no se fusionaron de vuelta a develop, hay conflictos sin resolver o no se siguió el flujo completo de GitFlow.	
			Rango del punteo	4 - 2.4	2.3 - 0	
1.5.4	Creación de Tag	4	Descripción	Se crearon tags en main para cada release. Los tags siguen versionamiento semántico. Los tags están correctamente posicionados en los commits de merge de release. Se alcanzó al menos la versión v2.0.0 como se requiere.	No se crearon tags, no siguen versionamiento semántico, están mal posicionados, no se alcanzó v2.0.0 o faltan tags para las releases realizadas.	

			Rango del punteo	4 - 2.4	2.3 - 0	
1.6	FLUJO HOTFIX	12				
1.6.1	Creación a partir de Main	12	Descripción	El flujo de HotFix está implementado completamente: (1) La rama hotfix se creó correctamente a partir de main (no de develop) (2) Sigue la nomenclatura 'hotfix/v#' o 'hotfix/descripcion-del-fix', (3) El hotfix se fusionó correctamente de vuelta a main sin conflictos, (4) Se creó un nuevo tag en main después del merge incrementando correctamente la versión (ej: de v2.0.0 a v2.0.1), siguiendo versionamiento semántico, (5) El hotfix también se fusionó a develop para mantener la corrección en desarrollo futuro, (6) La rama hotfix fue eliminada después de completar ambos merges, manteniendo el repositorio limpio.	El flujo de HotFix está incompleto o presenta errores significativos: no se creó a partir de main, la nomenclatura es incorrecta, no se fusionó correctamente a main o develop, hay conflictos sin resolver, no se creó tag o el tag no incrementa correctamente la versión, no se eliminó la rama hotfix o no existe evidencia de un hotfix funcional.	
			Rango del punteo	12 - 7.3	7.2 - 0	
1.7	MANUALES	6				

1.7.1	Manual Técnico con ERD	3	Descripción	El manual técnico está completo y en formato Markdown. Incluye: arquitectura del sistema, tecnologías utilizadas (lenguajes, frameworks, base de datos), Diagrama Entidad-Relación (ERD) de la base de datos, estructura de tablas y relaciones, guía de instalación y configuración, explicación de la API o endpoints si aplica. Está bien redactado y organizado. El ERD es claro y refleja correctamente la estructura de la base de datos.	El manual está incompleto, mal redactado, no incluye el ERD, el ERD no refleja la base de datos real, falta información técnica importante o no está en formato Markdown.	
			Rango del puntaje	3 - 1.8	1.7 - 0	
1.7.2	Manual de Usuario	3	Descripción	El manual de usuario está completo y en formato Markdown. Incluye: instrucciones paso a paso para cada funcionalidad, capturas de pantalla de todas las funcionalidades principales, explicación del flujo de uso de la aplicación. Está escrito en lenguaje claro para usuarios no técnicos. Las capturas son claras y relevantes	El manual está incompleto, mal redactado, faltan capturas de pantalla, las capturas no son claras, no cubre todas las funcionalidades, usa lenguaje muy técnico o no está en formato Markdown.	
			Rango del puntaje	3 - 1.8	1.7 - 0	
	Sub-Total de Puntos	51				
2	Conocimientos	49				

2.1	Manejo de Cuentas	12				
2.1.1	Login	3	Descripción	El sistema permite iniciar sesión correctamente validando credenciales. Muestra mensajes de error apropiados. La sesión se mantiene activa. Redirige al dashboard después del login exitoso.	El login presenta errores, no valida credenciales correctamente, permite acceso con datos incorrectos, no mantiene la sesión o no redirige apropiadamente.	
			Rango del punteo	3 - 1.8	1.7 - 0	
2.1.2	Registro de Usuario	3	Descripción	Permite registrar nuevos usuarios. Valida que el email no esté duplicado. Valida formato de email. Requiere confirmación de contraseña. Los datos se almacenan correctamente.	El registro presenta errores, permite duplicados, no válida formatos, no requiere confirmación o no almacena correctamente.	
			Rango del punteo	3 - 1.8	1.7 - 0	
2.1.3	Modificar Información Personal	3	Descripción	Permite editar información personal. Validar los nuevos datos. Requiere confirmación para cambios críticos. Los cambios persisten en la BD.	No permite editar todos los campos, no válida cambios, no requiere confirmación o no persiste correctamente.	
			Rango del punteo	3 - 1.8	1.7 - 0	
2.1.4	Validación de Acceso a Cuenta	3	Descripción	Valida que el usuario solo acceda a su cuenta. Implementa control de sesiones. Previene acceso no autorizado. Cierra sesión correctamente.	No valida correctamente el acceso, permite ver/editar datos de otros usuarios o presenta vulnerabilidades de seguridad.	

			Rango del punteo	3 - 1.8	1.7 - 0	
2.2	Gestión de Reseñas (CRUD)	15				
2.2.1	Agregar Reseña	5	Descripción	Permite agregar reseñas con título de película, calificación (1–5 estrellas), comentario y etiqueta/género. Valida que el título no esté vacío. Permite crear nuevas etiquetas o seleccionar las existentes. Los datos persisten en BD.	Presenta errores, no valida título vacío, no gestiona etiquetas adecuadamente, no guarda la calificación correctamente o no persiste datos.	
			Rango del punteo	5 - 3	2 - 0	
2.2.2	Eliminar Reseña	4	Descripción	Elimina reseñas de la interfaz y la BD. Implementa confirmación antes de eliminar. La eliminación es definitiva sin errores. La interfaz se actualiza inmediatamente.	No elimina correctamente, no solicita confirmación, genera errores o la interfaz no se actualiza.	
			Rango del punteo	4 - 2.4	2.3 - 0	
2.2.3	Modificar Reseña	6	Descripción	Permite modificar todos los campos (título, calificación, comentario, etiqueta). Mantiene validaciones. Permite cambiar etiquetas. Los cambios se reflejan en la interfaz y la BD inmediatamente.	Presenta errores, no permite editar todos los campos, no mantiene validaciones o no guarda correctamente.	
			Rango del punteo	6 - 3.6	3.5 - 0	

2.3	Funcionalidades Avanzadas	10				
2.3.1	Destacar Reseña	5	Descripción	Permite marcar/desmarcar una reseña como destacada/favorita. Las reseñas destacadas aparecen en la sección 'Destacadas' en la parte superior del dashboard. El estado persiste en BD. Se puede desmarcar en cualquier momento. Permite múltiples reseñas destacadas simultáneamente.	No funciona correctamente, las reseñas no aparecen en la sección correcta, el estado no persiste o no se puede desmarcar.	
			Rango del punteo	5 - 3	2 - 0	
2.3.2	Archivar Reseña	5	Descripción	Permite mover reseñas a la sección 'Archivadas'. Las reseñas archivadas desaparecen del dashboard principal pero son accesibles en la sección correspondiente. El estado persiste en BD. Se puede desarchivar. La interfaz diferencia claramente reseñas activas y archivadas.	No funciona, las archivadas siguen en vista principal, no existe sección de archivadas o el estado no persiste.	
			Rango del punteo	5 - 3	2 - 0	
2.4	Colaboración Entre Usuarios	8				
2.4.1	Compartir Reseña	4	Descripción	Permite seleccionar y compartir reseñas con otros usuarios registrados. Se puede buscar/seleccionar destinatarios. La reseña aparece en el perfil del receptor	No permite compartir correctamente, no se puede seleccionar usuarios, la reseña no aparece en el receptor o no mantiene información del propietario.	

				con indicador visual y nombre del propietario. Mantiene la autoría del propietario.		
			Rango del punteo	4 - 2.4	2.3 - 0	
2.4.2	Dashboard de Notificaciones	2	Descripción	Aparece en el dashboard de notificaciones del receptor. Muestra claramente que la reseña es compartida (icono/indicador). Muestra el nombre del propietario. Permite acceder directamente a la reseña compartida.	No existe dashboard, las reseñas compartidas no aparecen, no muestra el propietario o no tiene indicador visual.	
			Rango del punteo	2 - 1.2	1.1 - 0	
2.4.3	Sección de Compartidas	2	Descripción	El propietario ve en la sección 'Compartidas' todas las reseñas que ha compartido. Muestra con quién se compartió cada reseña. Permite identificar fácilmente el estado. La sección es accesible y organizada.	No existe la sección, no muestra las reseñas compartidas, no indica con quién se compartieron o es confusa.	
			Rango del punteo	2 - 1.2	1.1 - 0	
2.5 Preguntas		4				
2.5.1	Pregunta 1	1	Descripción	Respuesta Correcta	Respuesta Incorrecta	
			Rango del punteo	1	0	
2.5.2	Pregunta 2	1	Descripción	Respuesta Correcta	Respuesta Incorrecta	

			Rango del punteo	1	0	
2.5.3	Pregunta 3	1	Descripción	Respuesta Correcta	Respuesta Incorrecta	
			Rango del punteo	1	0	
2.5.4	Pregunta 4	1	Descripción	Respuesta Correcta	Respuesta Incorrecta	
			Rango del punteo	1	0	
	Sub-Total de Puntos	51				
	Total	100				

Comentarios Generales

Apartado para comentar si hubo algún inconveniente durante la calificación o comentario.